

Data Analytics

Big Data Analytics und Cloud Platforms

Prof. Stefan Keller, FS26

Das ist der erste von zwei Teilen (diese und nächste Woche) zu Big Data Analytics und Cloud Platforms

- Big Data Analytics
- Analytical Processing-Technologien: In-Memory und Column Stores
- Verteilte System-Technologien und Tools: MapReduce und Apache Spark
- Cloud Computing und Cloud Platforms

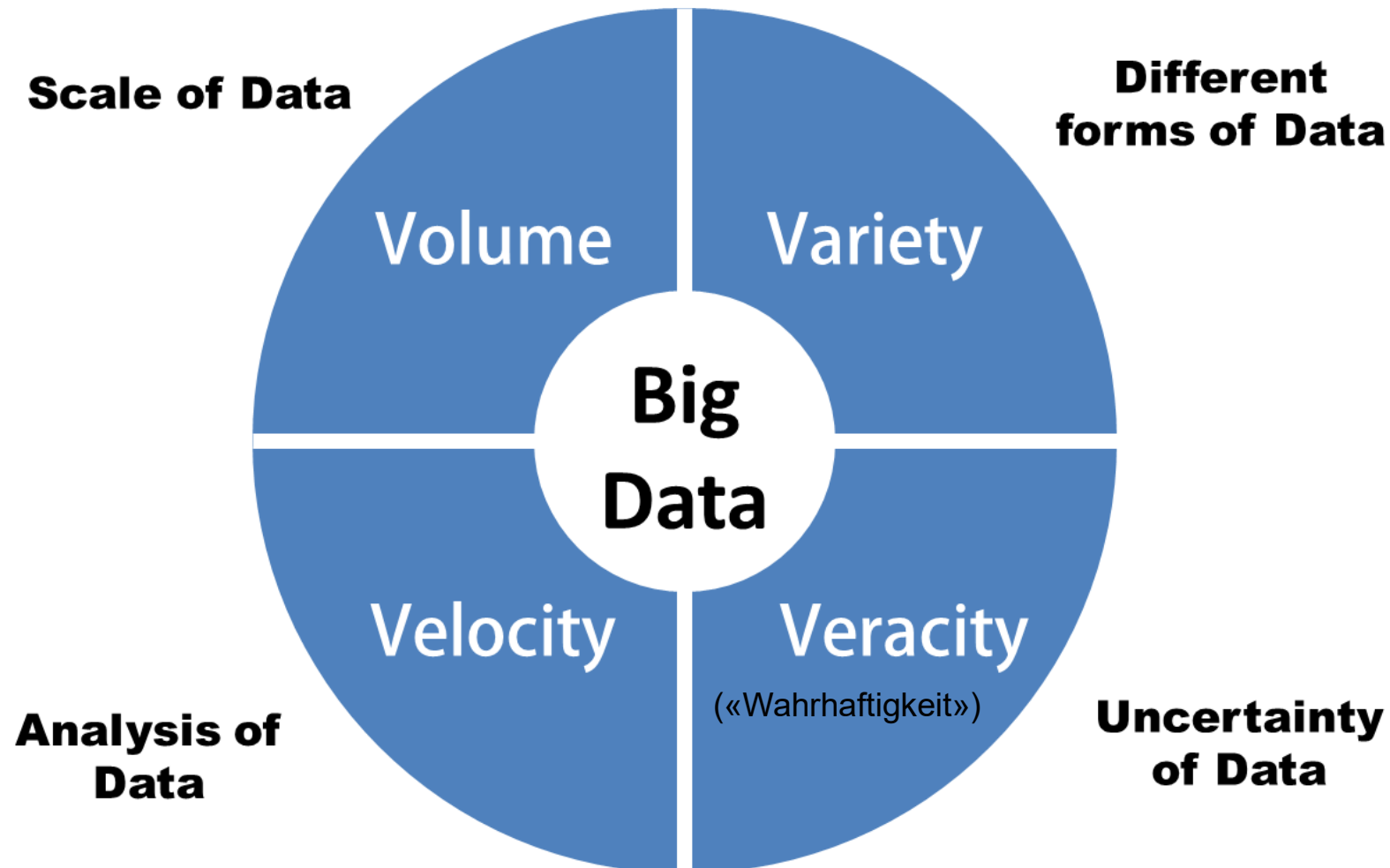
- Sie...
 - wissen, was mit Big Data Analytics gemeint ist
 - kennen einige Analytical Processing Technologien
 - kennen einige verteilte System-Technologien
 - kennen insbesondere das Apache Spark Framework
 - Kennen die Vor- und Nachteile von Cloud-Plattformen

BIG DATA ANALYTICS

Was ist Big Data?

- Zu "Big Data" gib es keine einheitliche Definition; wird häufig auch als Sammelbegriff verwendet für (Real-Time-)Analyse massiver Datenmengen
- Definition von Wikipedia: *Datenmengen, die zu gross, zu komplex, zu schnelllebig oder zu schwach strukturiert sind, um sie mit manuellen und herkömmlichen Methoden der Datenverarbeitung auszuwerten.*
- Eine der meistzitierten Definitionen: **Die «4 V's»:**
 - Volume, Variety, Velocity, Veracity → siehe nächste Folien
- Seit 2015 auch **«Die 5 V's»**
 - Zusätzlich mit (Business) Value, d.h. Erkenntnisgewinn, der zu besseren Entscheidungen oder Profit führt

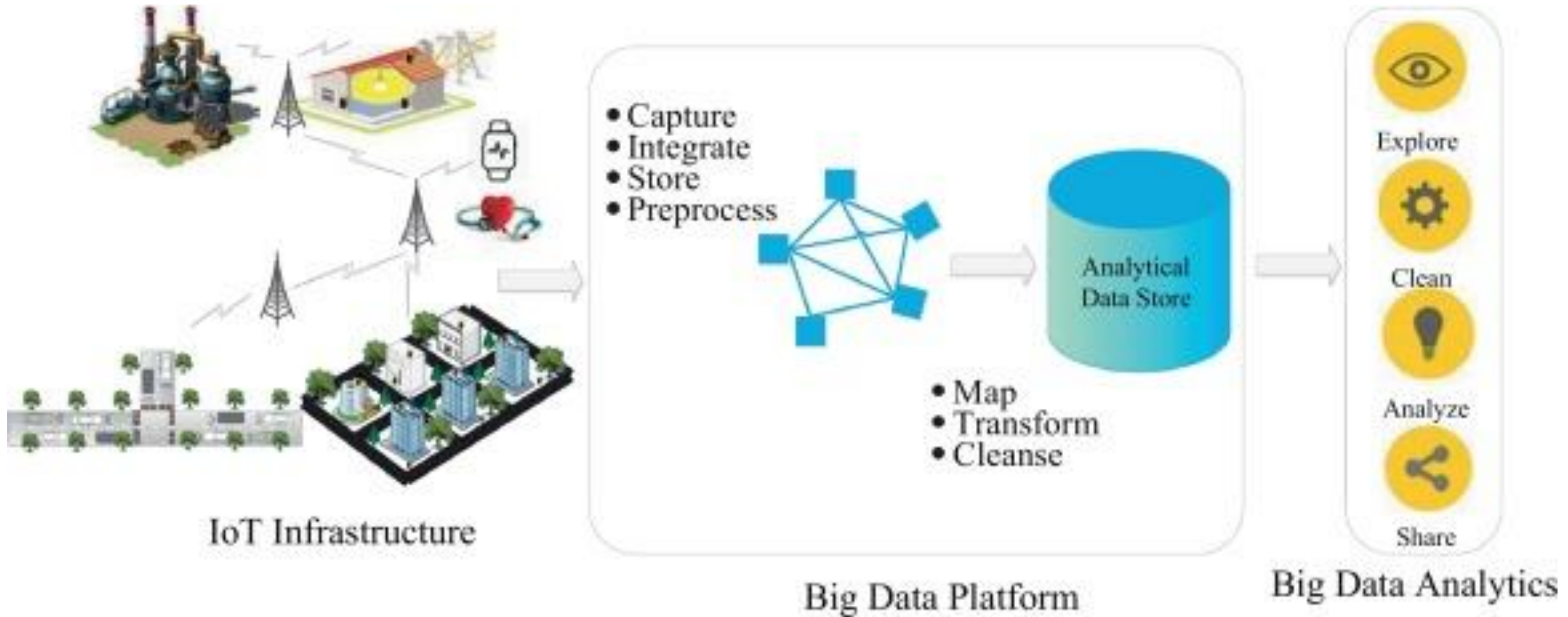
Die 4 V's von Big Data



Was ist Big Data? Eigene Definition (Keller, 2020)

- Small Data:
 - Daten haben im Memory Platz. Das ist bei der Programmiersprache R und den meisten Python Jupyter Notebooks (Pandas DataFrames) der Fall!
- Medium Data:
 - Daten haben auf Disk eines Laptops / Servers Platz aber nicht im Memory; z.B. Apache Spark, Python mit DuckDB
- Big Data:
 - Daten haben auch nicht auf Disk & Memory eines einzelnen Laptops / Servers Platz und verlangen nach verteilten Systemen

Big Data-Beispiele: Internet of Things



Quelle: "The role of big data analytics in Internet of Things" <https://www.sciencedirect.com/science/article/abs/pii/S1389128617302591>

Big Data-Beispiele: Point Clouds (Lidar-Daten)



Big Data-Beispiele: Luftbilder/Satellitenbilder

 Schweizerische Eidgenossenschaft
Confédération suisse
Confederazione Svizzera
Confederaziun svizra
In Zusammenarbeit mit den Kantonen

Ort suchen oder Karte hinzufügen:
Q z.B. Bundesplatz 1 Bern, 46.7 7.5, Lärmkarte ...

Probieren Sie test.map.geo.admin.ch aus Vollbild Problem melden Hilfe Mobile Version [DE](#) [FR](#) [IT](#) [EN](#) [RM](#)

▼ Menü öffnen



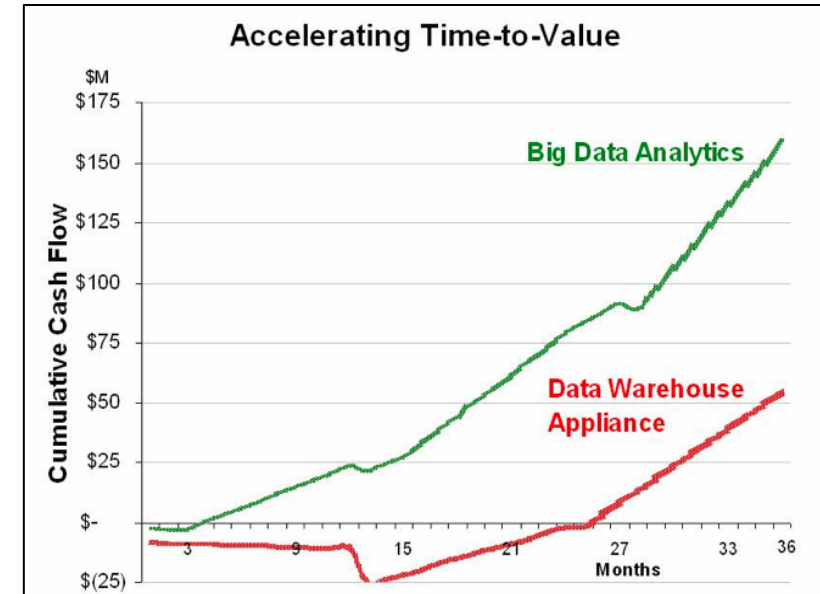
Hintergrund

© Daten: CNES, Spot Image, swisstopo, NPOC
geo.admin.ch Copyright & Datenschutzerklärung

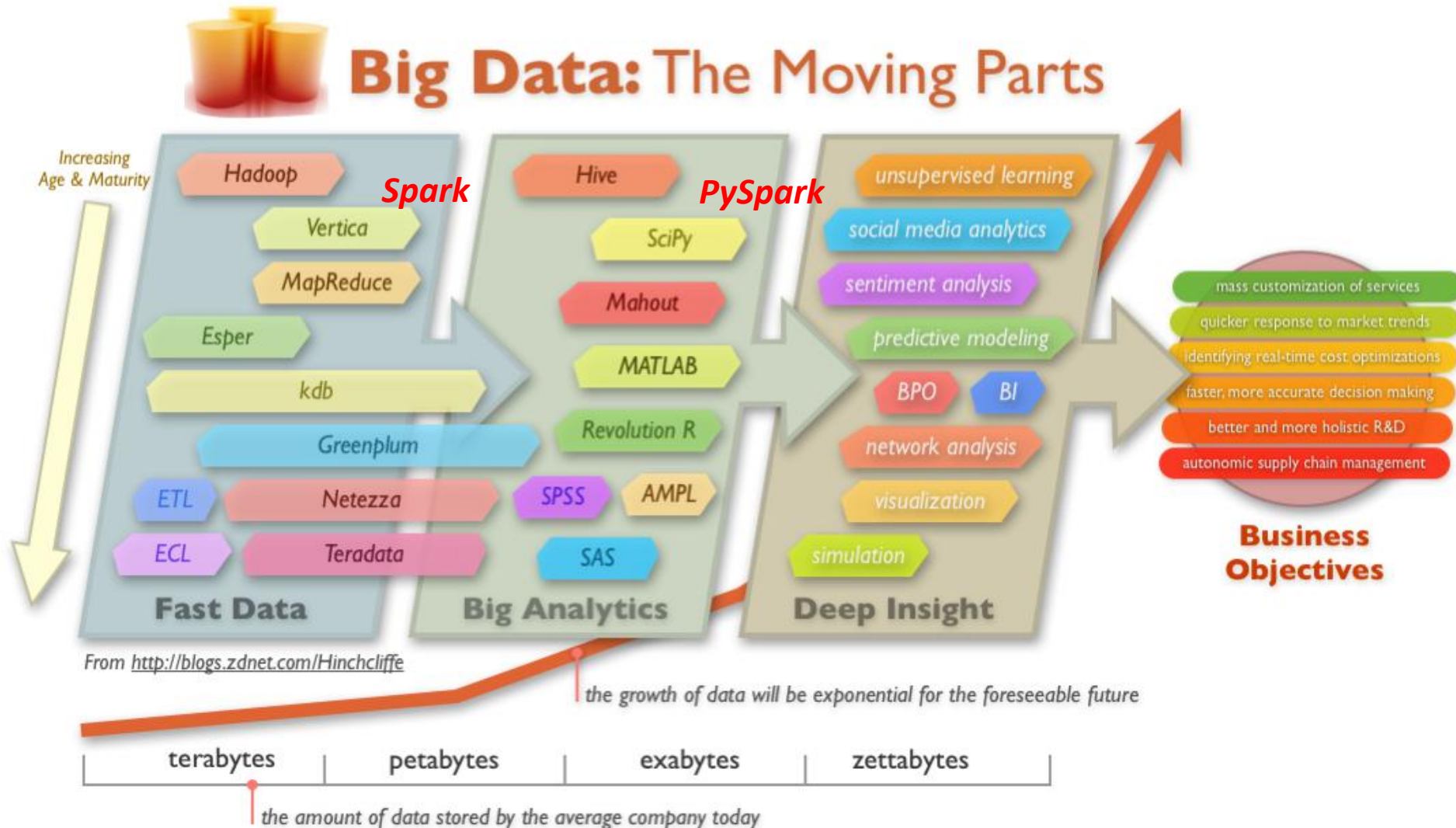
20 km CH1903+ / LV95 Koordinaten (m): 2'633'850.0, 1'232'275.0

Big Data Analytics

- Big Data ist oft real-time wozu traditionelle Data Warehouse-Architekturen ungeeignet sind
- Lösungsansatz 1
 - Scaling Up
 - In-Memory Column Stores
- Lösungsansatz 2
 - Scaling Out
 - “Shared nothing” Architektur mit Parallel Processing
 - Verteilte Systeme (engl. "Distributed Systems")



Big Data Technology



Diskussion: Was ist Big Data?

- Blogbeitrag von Jordan Tigani (*), Zusammenfassung:
 - Die meisten Daten sind nicht sehr gross (<100 GB)
 - Wenn doch, werden sie wahrscheinlich nicht alle auf einmal abgefragt
 - Selbst die Daten auf einmal abgefragt werden, passen sie wahrscheinlich immer noch auf einen einzelnen Rechner
 - Wenn nicht, horten Sie vielleicht Daten! Trimmen oder aggregieren Sie sie.
- (*) "Big Data is Dead" (2023) by Jordan Tigani, DuckDB Maintainer (Open Source); vorher bei Google BigQuery: <https://motherduck.com/blog/big-data-is-dead/>

Fazit: Natürlich ist Big Data nicht tot. Aber schaut besser hin...

Dann lasst uns nun verteilte Systeme näher anschauen:

In-Memory Column Stores

Datenstrukturen / Indexe

ANALYTICAL PROCESSING- TECHNOLOGIEN

In-Memory Column Stores

- Column Store = Spaltenorientiertes DBMS
 - Noch bessere Komprimierung und Indexierung möglich, da pro Spalte
- In-Memory Store
 - DBMS, das nur den Arbeitsspeicher als Datenspeicher nutzt
 - Problem Persistenz! Lösung z.B. mit logischem WAL oder mit Snapshots (= "Eventuelle Persistenz")
- Oft kombiniert!
- Vorteile:
 - Schnelle OLAP-Lese-Anfragen (10 bis 100 mal schneller als Row Stores)
- Nachteile:
 - Column Stores haben langsame Updates, da sämtliche "Columns" betroffen sind
 - Darum eher nicht geeignet für "write-heavy" (OLTP)

Spaltenorientierte DBMS (Auswahl)

- Oracle ab Version 12:
 - Spaltenorientierte In-Memory-Erweiterung
- MS SQL Server:
 - Spaltenorientierte Erweiterung (nicht In-Memory)
- PostgreSQL:
 - Spaltenorientierte Erweiterungen (nicht In-Memory) pg_duckdb, Citus, TimescaleDB mit spaltenorientierter Auslagerung
- SAP Hana:
 - Spaltenorientiertes in-memory DBMS (Trivia: Mindest-HW-Konfiguration ist angeblich ca. 500 GB Memory!)
- DuckDB
 - Spaltenorientiertes, in-process DBMS, Open Source (kann über RAM hinaus skalieren)

- «Caching»: Materialized Views und eigene Strukturen (wie besprochen)
- Indexe: Bitmap-Indexe, u.a. für Data Warehouses
- Parallel Query Processing: Ausnutzung von CPUs / GPUs
- "Vectorized" Query Processing Style vs. "Volcano" Style:
 - "Volcano Style":
 - Jede Operation verarbeitet ein Tupel (Zeile) vollständig, bevor das nächste Tupel bearbeitet wird (Tupel-at-a-Time)
 - Einfach zu Implementieren
 - Führt zu Overhead, da für jedes Tupel Funktionsaufrufe und Kontextwechsel erforderlich sind
 - "Vectorized Style":
 - Verarbeitet Daten in Batches (Stapeln) von Vektoren (Arrays von Werten) - was bei Column Stores "natürlich" ist
 - Reduziert Overhead durch geringere Funktionsaufrufe und Kontextwechsel
 - Kann Vorteile aus modernen CPU-Architekturen ziehen, wie "Single Instruction, Multiple Data" (SIMD)
 - (Achtung: Hat nichts mit "Vector Stores" zu tun, die mit ChatGPT & Co. bekannt wurden)

VERTEILTE SYSTEM-TECHNOLOGIEN UND TOOLS

- Single-Machine – lokal, RAM-limitiert
 - Pandas – eager, Python-Standard
 - Polars – lazy + eager, Rust-Kern, deutlich schneller
 - DuckDB – eingebettete In-Process-OLAP-Engine
- Hybrid (Single-Machine → Cluster)
 - Dask – Pandas-API, optional auf Cluster skalierbar
 - Ray / Modin – parallele Python-Workloads
- Distributed Compute Engine (Open Source)
 - Apache Hadoop (veraltet, Legacy) – MapReduce-basiert, disk-intensiv, Vorläufer von Spark
 - Apache Spark – on-premise + Cloud, Batch + Streaming
 - Apache Flink – Stream-first, Echtzeit-Verarbeitung
- Cloud-Plattformen & Data Warehouses
 - Databricks – kommerzielle Plattform auf Spark
 - Snowflake – kommerzielles Cloud Data Warehouse
 - Google BigQuery – serverless, columnar, Google Cloud
 - Amazon Redshift – AWS Data Warehouse, columnar
 - ClickHouse – open source, columnar

Introduction

Architecture

MAPREDUCE - OVERVIEW

MapReduce

- Ursprünglich von Google (Dean & Ghemawat, 2004)
- Ein einfaches Programmiermodell
- Ein funktionales Modell
- Für gross angelegte Datenverarbeitung:
 - Nutzt eine grosse Anzahl von Standardcomputern
 - Führt Prozesse in verteilter Weise aus
 - Bietet hohe Verfügbarkeit

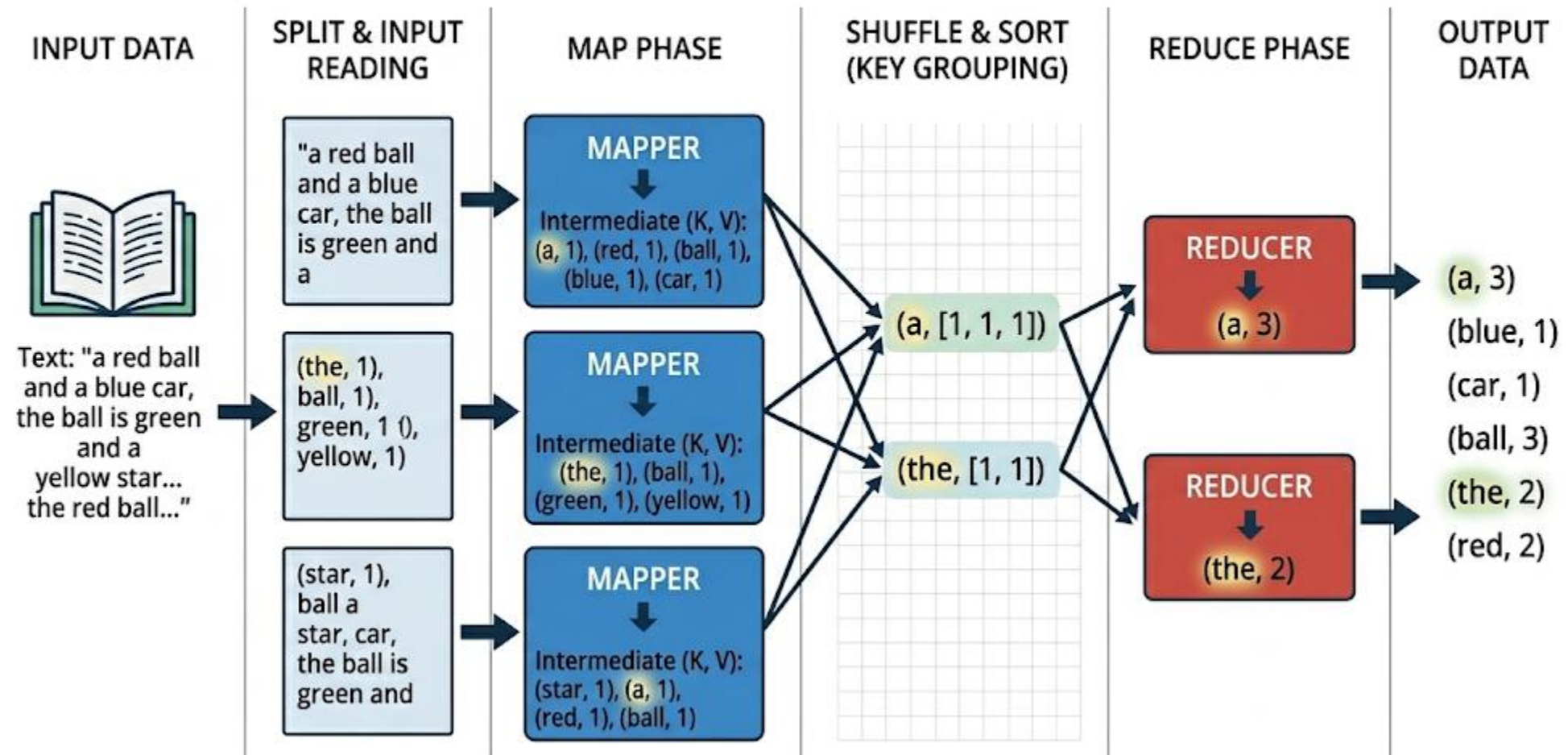
MapReduce: Motivation

- Viele Anforderungen an die Verarbeitung sehr grosser Datenmengen
- Bestimmte gemeinsame Themen für diese Anforderungen
 - Es werden viele Maschinen benötigt (Skalierung)
 - Zwei Grundoperationen für die Eingabe
 - Abbilden (map)
 - Reduzieren (reduce)
- Vereinfacht umfangreiche Berechnungen, die zu diesem Modell passen
- Ermöglicht es dem Benutzer, sich auf das Problem zu konzentrieren, ohne sich um Details zu kümmern
- Computerarchitektur nicht sehr wichtig

MapReduce: Map + Reduce

- Funktionen, die aus funktionalen Programmiersprachen wie (z.B. Lisp) entlehnt sind
- Map
 - Verarbeitet ein Schlüssel/Wert-Paar, um Zwischen-Schlüssel/Wert-Paare zu generieren
- Reduce
 - Fasse alle Zwischenwerte zusammen, die mit demselben Schlüssel verknüpft sind
- Partition
 - Default: $\text{hash}(\text{key}) \bmod N$
 - Gut ausbalanciert

MapReduce: Beispiel Wörter "the" und "a" zählen



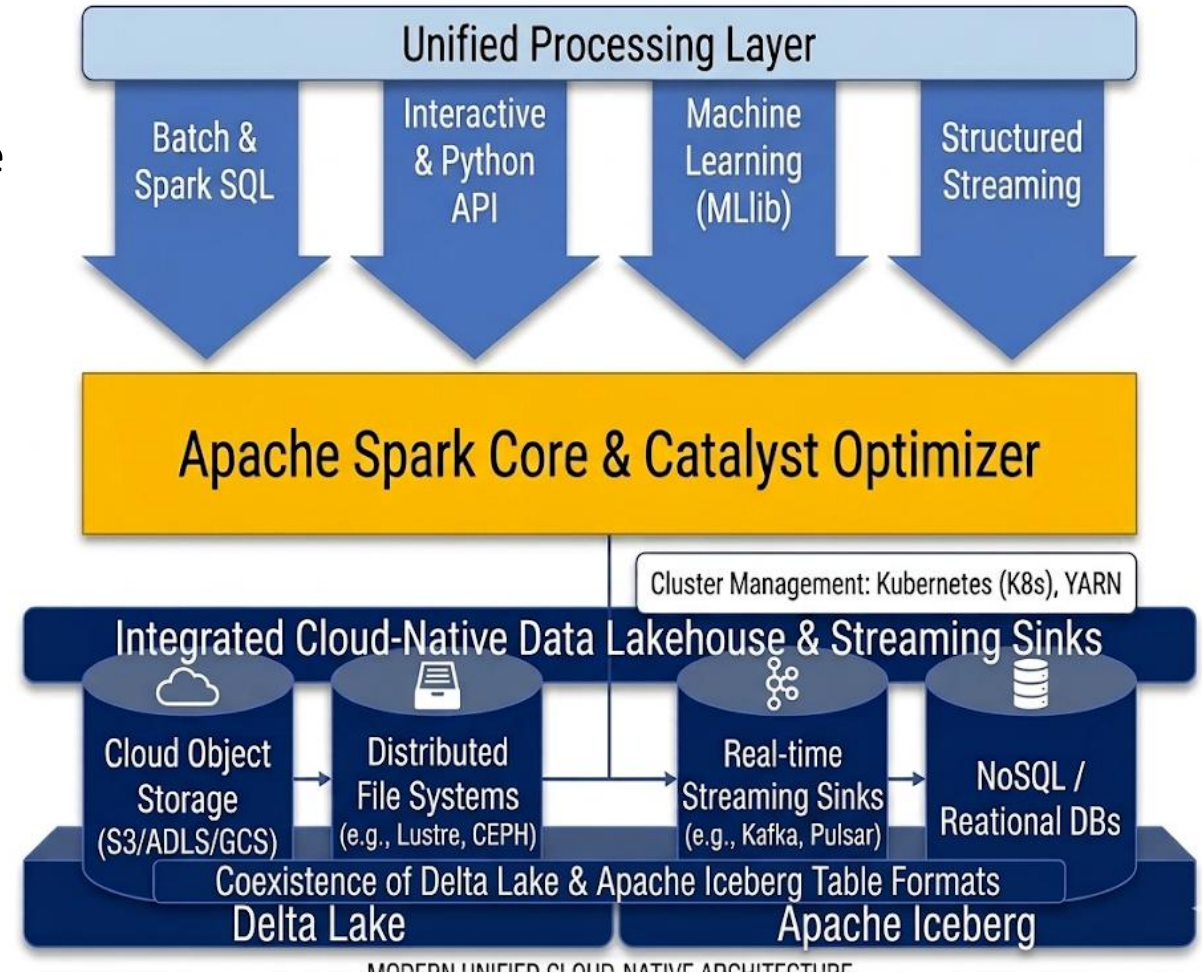
- Nachteile
 - Der Master ist ein zentraler "Point-of-Failure" und Engpass für die Skalierbarkeit
 - Latenz beim Öffnen von Tausenden von Dateien
- Diskussion MapReduce
 - War lange ein Kernkonzept in der Big-Data-Welt, insbesondere in Hadoop-Umgebungen
 - Ist noch in bestimmten Szenarien wichtig, wie z.B. Massive Batch-Jobs, Log-Analysen, verteilte Berechnungen ohne Echtzeitanforderungen
- Andere Technologien sind inzwischen populärer: Siehe nachfolgend

APACHE SPARK

- Ein Open-Source Big-Data-Framework mit verteilter Architektur
- Eine schnelle Engine (In-Memory Computing) für die Verarbeitung grosser Datenmengen
- Fokussiert auf
 - Iterative Programme (ML)
 - Interaktive Abfragen
 - Vereinheitlichung von Echtzeit- und Stapelverarbeitung
- 2014 als Apache Top-Level Project (Open Source)

Apache Spark: Architektur

- Unified Engine: Stack für Batch, Real-time Streaming, SQL, Spark ML und Graph-Analyse.
- Separation of Compute & Storage: Cloud-native Architektur, die Rechenleistung flexibel von Objektspeichern (S3, ADLS) und Lakehouse-Formaten (Delta Lake, Iceberg) trennt.
- Optimierte Performance: Nutzung des Catalyst Optimizers für effiziente Abfragepläne.
- Hierarchische Struktur: Ein zentraler Driver-Prozess steuert die Aufgabenverteilung an verteilte Executoren (Worker-Nodes).
- Modernes Management: Ressourcensteuerung primär über Kubernetes (K8s) sowie entkoppelte Client-Verbindungen via Spark Connect.



Apache Spark: Viele Anbindungen und Erweiterungen

- Geschrieben in Scala mit Spark API für Scala, Java, Python (PySpark) und R (SparkR)
- Alle Verarbeitungs-Komponenten in Spark nutzen DataFrames als Abstraktion (früher Resilient Distributed Datasets RDDs)
- Spark SQL
- DataFrame/Dataset-API

Apache Spark: Spark SQL

- Bietet die Möglichkeit, die Spark-Datensätze über JDBC-API freizugeben und SQL-ähnliche Abfragen auf Spark-Daten mit herkömmlichen BI- und Visualisierungswerkzeugen auszuführen.
- Ermöglicht es den Benutzern, ihre Daten aus verschiedenen Formaten zu ETL zu machen, sie umzuwandeln und sie für Ad-hoc-Abfragen freizugeben.
- Ist funktional gleichwertig zum DataFrame/Dataset-API
- Beispiel: `spark.sql("SELECT ...")`
- Datenquellen
 - Datenbanken
 - Datei-Format Parquet: Kolonnen-orientiertes Dateiformat
 - Datei-Formate JSON, Protobuf, Avro, ORC (column-based wie Parquet)

Apache Spark: DataFrame/Dataset-API

- Unified API – Seit Spark 2.0 vereint die DataFrame/Dataset-API Batch- und Streaming-Verarbeitung unter einer einheitlichen Schnittstelle.
- Typsicherheit – Das Dataset `[T]` ergänzt den DataFrame um Compile-Zeit-Typsicherheit in Scala/Java.
- Strukturierte Daten – analog zu einer SQL-Tabelle oder einem Pandas DataFrame, mit explizitem Schema (Spaltenname + Datentyp), das entweder inferiert oder manuell definiert wird.
- Immutabilität – DataFrames sind unveränderlich; jede Transformation erzeugt einen neuen DataFrame, der ursprüngliche bleibt unberührt.
- Lazy Evaluation – Transformationen werden erst ausgeführt, wenn eine Action (z. B. `show()`, `collect()`) aufgerufen wird.
- Verteilung & Partitionierung – Intern sind DataFrames in Partitionen aufgeteilt, die parallel auf dem Cluster verarbeitet werden; `repartition()` und `coalesce()` steuern die Verteilung.
- Achtung: DataFrame ist ein Konzept, Pandas und Spark DataFrames Implementierungen.

Apache Spark: DataFrame/Dataset-API Beispiel

```
from pyspark.sql import SparkSession

spark =
    SparkSession.builder.appName('Beispiel').getOrCreate()
data = [(1, 'Alice'), (2, 'Bob')]
df = spark.createDataFrame(data, ['ID', 'Name'])
df.show()
```

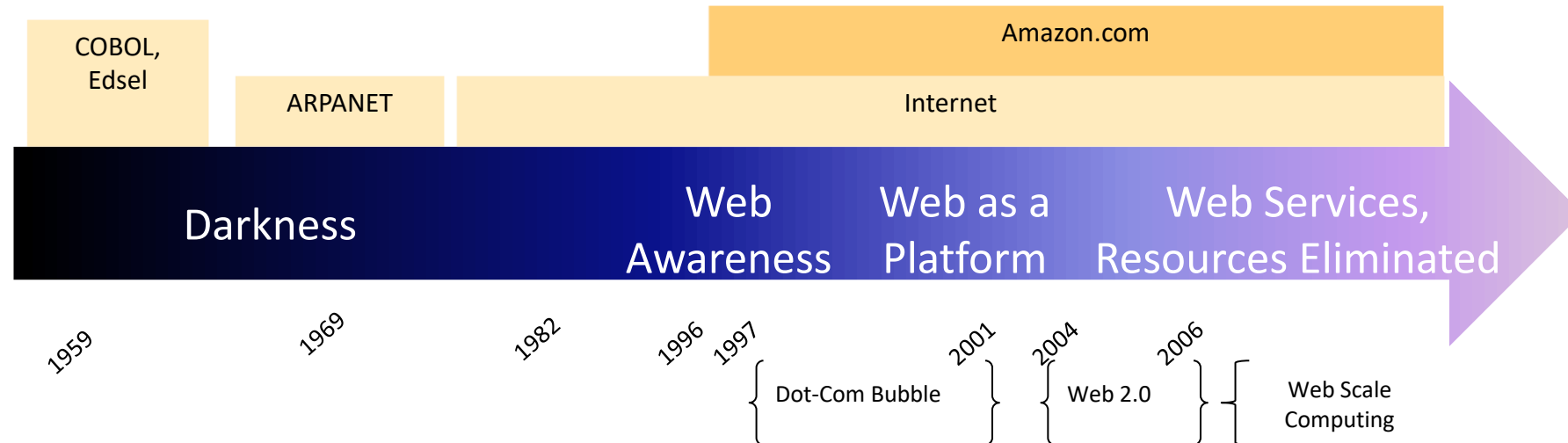
Vergleich

Merkmal	Relationale Tabelle	Pandas	Polars	DuckDB
Speicherort	Datenbank-Server	Arbeitsspeicher	Arbeitsspeicher	Arbeitsspeicher / Disk
Schema	Strikt, vorab def.	Inferiert, änderbar	Strikt, unveränderlich	Strikt, vorab def.
Auswertung	Deklarativ (SQL)	Eager (sofort)	Lazy + Eager	Deklarativ (SQL)
Veränderbarkeit	Transaktional (ACID)	Mutable	Immutable	Immutable (Abfragen)
Skalierung	Vertikal / horizontal	RAM-limitiert	RAM-limitiert, effizienter	Über RAM hinaus
Parallelität	Ja (server-seitig)	Nein (single-thread)	Ja (multi-thread)	Ja (multi-thread)
Sprache / Interface	SQL	Python	Python, Rust	SQL, Python, R
Indexierung	Primär-/Fremdschlüssel	Expliziter Index	Kein Index	Kein expliziter Index
Fehlende Werte	NULL	NaN / None	Null (typisiert)	NULL
Typensystem	Strikt	Schwach (object)	Strikt	Strikt

CLOUD COMPUTING

Etwas Geschichte zu Cloud Platforms bis 1960–2000 (1)

- Cloud Platforms



- Man beachte die direkte Assoziation von "Distributed" mit "Cloud" (wobei "cloud" ja einfach "der Computer eines anderen ist" 😊)
- Quellen: "The Obligatory Timeline Slide" (von Mike Culver, Amazon)

Geschichte zu Cloud Platforms und Cloud Databases (2)

- Ab 2000: Virtualization, Cloud Computing, “Everything as a Service” (XaaS), Edge Computing
- 2000–2003 — ASP-Ära, Salesforce als SaaS-Beweis
- 2003–2006 — Google-Paper (GFS, MapReduce, Bigtable) + AWS-Gründung
- 2007–2010 — NoSQL-“Revolution”, CAP-Theorem, erste DBaaS (RDS)
- 2010–2015 — NewSQL, Cloud Data Warehouses (Redshift, BigQuery), Spark
- 2015–2019 — Compute/Storage-Trennung, Snowflake, Serverless, HTAP
- 2019–2022 — Data Lakehouse, Open Formats (Iceberg/Delta), Data Mesh
- 2022–2024 — Vector Databases, RAG, Edge-DBs, AI-Integration
- 2025–... — Schlägt der Pendel schlägt zurück Richtung On-Premises?
- Fazit:
 - SQL wurde mehrfach totgesagt — und lebt jedes Mal stärker wieder auf.
 - Die Revolution war nicht NoSQL, sondern die Trennung von Compute und Storage als Architekturprinzip.
- Die Frage ist Cloud vs. On-Prem, sondern ...
 - die richtige Workload am richtigen Ort — das ist die eigentliche Kernkompetenz im modernen Data Engineering.

- Definition "Cloud Computing":

"Ein Modell für den bequemen, bedarfsorientierten Netzzugang zu einem gemeinsam genutzten Pool konfigurierbarer Rechenressourcen (z.B. Netze, Server, Speicher, Anwendungen und Dienste), die mit minimalem Verwaltungs- oder Interaktionsaufwand mit dem Dienstanbieter schnell bereitgestellt und freigegeben werden können."

- Quelle: NIST

- "RAM is king": Das Random-Access Memory ist die primäre Datenquelle und die Festplatte ist Fallback
- Netzwerk-Durchsatz wird immer besser
- Multi-Core CPUs sind alltäglich ("commodity" = normale Ware)
- Immer mehr Unternehmen werten grosse Datenmengen aus
- Nebst Verarbeitung grosser Datenmengen (en: volume) wird auch die Verarbeitungsgeschwindigkeit (en. velocity) wichtig

CLOUD PLATFORMS

Wieso Cloud?

Vorteile und Nachteile von Clouddanwendungen?



https://www.ingenieur.de/wp-content/uploads/2018/05/Cloud-computing_c_panthermedia_maximsamos_B31715283_1000x749-e1527514653309-980x490.jpg

Cloud-Anwendungen vs. on-premise ("on prem")

- Im Folgenden ein Vergleich von Cloud-Anwendungen mit "Third-Party on-premise" DBs (Third-Party = auf Application Hosting spezialisierte Firmen):
- Vorteile für beide: Weniger IT-spezialisiertes Personal nötig
- Vorteile Cloud-Anwendungen gegenüber "on prem":
 - Skalierbarkeit, Flexibilität, geringere Initial- und Vorlaufkosten
 - Für Prototyping (Hochfahren, Testen, Wegwerfen) schneller und meist günstiger
 - Für grosse Unternehmen evtl. sinnvoll, da das Änderungen an Architektur und Prozessen erzwingt
- Nachteile von Cloud-Anwendungen gegenüber "on prem":
 - Man muss sich - entgegen der Werbung(!) - immer noch um Technik, Integration und Betrieb kümmern
 - Verlust der physischen Kontrolle; keine Garantie des Datenschutzes (GDPR)
 - Schnelle Migration ("Lift and Shift") in die Cloud mag zunächst günstiger sein als on-prem; dann wird es teurer, denn oft ist es notwendig, Anwendungen enger an die Cloud-nativen Prinzipien und Technologien anzupassen (z.B. AWS Lambdas), um die Vorteile ausschöpfen zu können
 - Fehlende Standards, z.B. für die Übertragung von Daten von einer Cloud in die andere Cloud
 - Folge der letzten beiden Punkte: (Noch mehr) Bindung an den Cloud-Anbieter (vendor-lockin)

- U.a. für Data Analysis und Data Storage etc.:
 - Amazon Web Services (AWS) – Der Markführer
 - Microsoft Azure - Für OST-Studierende mit Account: <https://portal.azure.com/>
 - Google Cloud Platform (GCP) – auch sehr verbreitet
 - Google Earth Engine (GEE) – siehe Übungen nächste Woche
- dann:
 - IBM Bluemix
 - DigitalOcean
 - Snowflake
 - Apache Spark (Open Source)
 - etc. (Oracle fehlt hier etwas bei Data Analytics)
- Mehr dazu nächste Woche

ABSCHLUSS

- Big Data Analytics
 - Die 4 bzw. 5 V's von Big Data - aber: nicht alles ist Big Data)
 - Small / Medium / Big Data
- Analytical Processing-Technologien:
 - In-Memory und Column Stores und Optimierungen (materialisierte Strukturen, Parallel Computing, Indexe, Vector Style Processing)
- Verteilte System-Technologien und Tools:
 - MapReduce und Apache Spark
- Cloud Computing und Cloud Platforms
 - Vorteile und Nachteile

- Übungen:
 - Jupyter Notebook mit Python/PySpark
- Nächste Woche (VoWo 06):
 - Big Data Analytics und Distributed Systems (Fortsetzung)
 - Spatial Data Analytics: Rasterdaten-Analyse mit dem "Platform-as-a-Service" Google Earth Engine